Implementation of selected quantum algebra and number theory algorithms

Mateusz Hurkała

Master thesis
Tutor: D.Sc. Krzysztof Turowski

Jagiellonian University
Department of Mathematic and Computer Science
Kraków 2026

Contents

1 Theoretical Foundations of Quantum Computation **2**

1.1 Introduction . . . 2

1.2 Mathematical Prerequisites . . . 3

1.2.1 Hilbert Spaces and Dirac Notation . . . 3

1.2.2 Inner and Outer Products . . . 3

1.2.3 Linear Operators . . . 4

1.3 The Qubit . . . 4

1.3.1 Computational Basis . . . 4

1.3.2 Superposition . . . 5

1.3.3 The Bloch Sphere . . . 5

1.4 Composite Systems and Tensor Products . . . 6

1.5 Quantum Entanglement . . . 7

1.5.1 The Bell States . . . 7

1.6 Quantum Logic Gates and Circuits . . . 7

1.6.1 Single-Qubit Gates . . . 8

1.6.2 Multi-Qubit Gates: The CNOT Gate . . . 8

1.7 Universal Quantum Computation . . . 9

1.8 Theoretical Machine and Memory Models . . . 9

1.8.1 The Quantum Turing Machine (QTM) . . . 9

1.8.2 Quantum Random Access Memory (QRAM) . . . 10

1.9 Computational Complexity and Quantum Advantage . . . 11

1.9.1 Definitions of Complexity Classes . . . 11

1.9.2 Proofs of Inclusion . . . 12

Chapter 1

Theoretical Foundations of Quantum Computation

1.1 Introduction

The theoretical limits of computation have traditionally been bound by the laws of classical physics. The Turing Machine, the abstract mathematical model that underpins all modern classical computers, operates on a deterministic tape of zeros and ones. For decades, the growth of computational power followed Moore's Law, doubling the density of transistors on integrated circuits approximately every two years. However, as transistor features approach the atomic scale (sub-5nm), classical deterministic behavior begins to break down due to quantum mechanical effects such as electron tunneling and thermal noise.

In the early 1980s, physicists Paul Benioff [Ben80] and Richard Feynman [Fey82] roposed a paradigm shift. Feynman famously observed that simulating quantum mechanical systems on classical computers appears to incur an exponential overhead in computational resources. He conjectured that a computer itself governed by quantum mechanics could simulate physical systems efficiently. This idea was formalized by David Deutsch in 1985, who defined the Universal Quantum Turing Machine [Deu85], proving that a quantum computer could strictly simulate any physical process.

Unlike classical bits, which exist in a definite state of 0 or 1, quantum bits (qubits) leverage two fundamental quantum phenomena: *superposition* and *entanglement*.

These properties allow quantum algorithms to explore a vast Hilbert space of computational states simultaneously. The discovery of polynomial-time quantum algorithms for problems that are exponentially hard for classical computers-most notably Shor's algorithm for integer factorization [Sho94]-demonstrated that quantum computing is not merely a physical curiosity, but a distinct computational complexity class with the potential to break modern cryptographic infrastructure.

This chapter establishes the rigorous mathematical and physical frameworks necessary to understand these algorithms. We begin with linear algebra and the postulates of quantum mechanics, move to the definition of the qubit and composite systems, and conclude with the circuit model of quantum computation.

1.2 Mathematical Prerequisites

Quantum mechanics is formulated in the language of linear algebra. Understanding quantum algorithms requires fluency in the properties of complex vector spaces, inner products, and operator theory.

1.2.1 Hilbert Spaces and Dirac Notation

The state of an isolated quantum system is described by a unit vector in a complex Hilbert space, denoted $\mathcal{H}$.

A Hilbert space is a vector space equipped with an inner product that is complete with respect to the norm induced by that inner product. For the finite-dimensional systems relevant to quantum computing (such as a system of n qubits), the Hilbert space is isomorphic to $\mathbb{C}^{2^n}$.

We employ the Dirac *bra-ket* notation standard in quantum physics:

- **Ket** ($|\psi\rangle$): A column vector representing a quantum state.
- **Bra** ($\langle\psi|$): The Hermitian conjugate (conjugate transpose) of the ket, representing a row vector in the dual space $\mathcal{H}^*$.

For a vector $\psi \in \mathbb{C}^d$, we write:

$$|\psi\rangle = \begin{pmatrix} z_1 \\ z_2 \\ \vdots \\ z_d \end{pmatrix}, \quad \langle\psi| = |\psi\rangle^\dagger = \begin{pmatrix} z_1^* & z_2^* & \dots & z_d^* \end{pmatrix} \tag{1.1}$$

where z^* denotes the complex conjugate of z.

1.2.2 Inner and Outer Products

The **inner product** of two state vectors $|\phi\rangle$ and $|\psi\rangle$ is a scalar $c \in \mathbb{C}$, denoted by the bracket $\langle\phi|\psi\rangle$. It satisfies the following properties:

1. Conjugate symmetry: $\langle\phi|\psi\rangle = \langle\psi|\phi\rangle^*$.
2. Linearity in the second argument: $\langle\phi| (a|\psi_1\rangle + b|\psi_2\rangle) = a\langle\phi|\psi_1\rangle + b\langle\phi|\psi_2\rangle$.
3. Positive definiteness: $\langle\psi|\psi\rangle \geq 0$, with equality if and only if $|\psi\rangle = 0$.

Two vectors are *orthogonal* if their inner product is zero: $\langle\phi|\psi\rangle = 0$. The norm of a vector is defined as $||\psi|| = \sqrt{\langle\psi|\psi\rangle}$. To represent a valid physical quantum state, a state vector must be normalized to unity ($||\psi|| = 1$).

The **outer product** of $|\psi\rangle$ and $|\phi\rangle$ is an operator (matrix) $A = |\psi\rangle\langle\phi|$. Applying this operator to a vector $|\xi\rangle$ yields a new vector proportional to $|\psi\rangle$:

$$(|\psi\rangle\langle\phi|)|\xi\rangle = |\psi\rangle(\langle\phi|\xi\rangle) = c|\psi\rangle \tag{1.2}$$

where $c = \langle\phi|\xi\rangle$ is a scalar.

1.2.3 Linear Operators

Quantum evolution is described by linear operators acting on $\mathcal{H}$. An operator A is linear if $A(c_1 |\psi_1\rangle + c_2 |\psi_2\rangle) = c_1 A |\psi_1\rangle + c_2 A |\psi_2\rangle$.

Hermitian Operators

An operator A is *Hermitian* (or self-adjoint) if $A = A^\dagger$, where $A^\dagger$ is the conjugate transpose of A.

Hermitian operators are crucial because they represent physical observables (such as energy or spin). The Spectral Theorem states that for any Hermitian operator M on a finite-dimensional space, there exists an orthonormal basis of eigenvectors $\{|m_i\rangle\}$ with real eigenvalues m_i such that:

$$M = \sum_i m_i |m_i\rangle \langle m_i| \tag{1.3}$$

This decomposition implies that the outcome of a measurement of observable M is always one of the real values m_i.

Unitary Operators

An operator U is *unitary* if its inverse is its adjoint:

$$U^\dagger U = UU^\dagger = I \tag{1.4}$$

Unitary operators preserve the inner product: $\langle\psi| U^\dagger U |\phi\rangle = \langle\psi|\phi\rangle$. Consequently, they preserve the norm of state vectors. This property is essential for quantum computing, as it ensures that the sum of probabilities of all possible outcomes remains 1 during the evolution of the system.

All closed quantum systems evolve via unitary transformations.

1.3 The Qubit

The fundamental unit of quantum information is the *qubit*.

While a classical bit is physically realized by a system with two distinct stable states (e.g., high/low voltage), a qubit is a two-level quantum mechanical system.

1.3.1 Computational Basis

The mathematical representation of a single qubit is a vector in the two-dimensional Hilbert space $\mathbb{C}^2$.

We define an orthonormal basis $\{|0\rangle, |1\rangle\}$, known as the *computational basis*, corresponding to the classical bit values 0 and 1:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \tag{1.5}$$

1.3.2 Superposition

Unlike a classical bit, a qubit can exist in a linear combination of basis states, a phenomenon known as *superposition*. A general qubit state $|\psi\rangle$ is described by:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle \tag{1.6}$$

where $\alpha, \beta \in \mathbb{C}$ are complex probability amplitudes. The Born rule dictates that upon measurement in the computational basis, the probability of observing outcome $|0\rangle$ is $P(0) = |\alpha|^2$, and the probability of observing $|1\rangle$ is $P(1) = |\beta|^2$. The normalization condition requires:

$$|\alpha|^2 + |\beta|^2 = 1 \tag{1.7}$$

1.3.3 The Bloch Sphere

To visualize a single qubit, we can simplify the parameters. Since the global phase of a quantum state (a factor of $e^{i\gamma}$) has no observable physical consequence, we can rewrite the state $|\psi\rangle$ effectively using only two real degrees of freedom, θ and ϕ:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle \tag{1.8}$$

Here, $0 \leq \theta \leq \pi$ and $0 \leq \phi < 2\pi$.

This parameterization maps every pure qubit state to a unique point on the surface of a unit sphere in $\mathbb{R}^3$, known as the **Bloch Sphere**.

- The North Pole ($\theta = 0$) represents the state $|0\rangle$.
- The South Pole ($\theta = \pi$) represents the state $|1\rangle$.
- Points on the equator (e.g., $\theta = \pi/2, \phi = 0$) represent equal superpositions such as $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$.

While the Bloch sphere provides an excellent intuition for single-qubit gates (which correspond to rotations on the sphere), it does not generalize to multi-qubit systems due to entanglement.

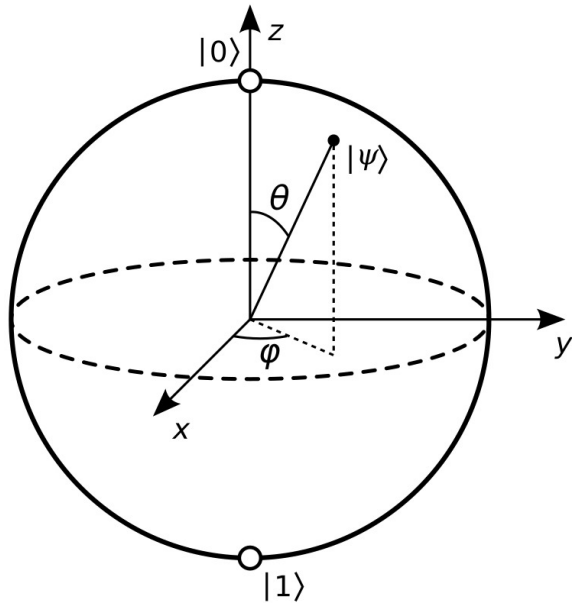

Figure 1.1: Bloch sphere: Source: *Wikipedia*.

1.4 Composite Systems and Tensor Products

While a single qubit provides the fundamental basis for quantum information, the true power of quantum computation emerges only when multiple qubits are combined.

To describe the state space of a multi-partite quantum system, we utilize the *tensor product* (or Kronecker product), denoted by $\otimes$.

If system A is described by a Hilbert space $\mathcal{H}_A$ of dimension N, and system B is described by a Hilbert space $\mathcal{H}_B$ of dimension M, the composite system AB resides in the tensor product space $\mathcal{H}_A \otimes \mathcal{H}_B$, which has dimension $N \times M$.

For two qubits, each residing in $\mathbb{C}^2$, the composite two-qubit state lives in $\mathbb{C}^2 \otimes \mathbb{C}^2 \cong \mathbb{C}^4$.

The computational basis for this composite system is formed by the tensor products of the individual basis states:

$$\{|0\rangle \otimes |0\rangle, |0\rangle \otimes |1\rangle, |1\rangle \otimes |0\rangle, |1\rangle \otimes |1\rangle\} \tag{1.9}$$

For brevity, we often omit the tensor product symbol and write these basis states as $|00\rangle, |01\rangle, |10\rangle$, and $|11\rangle$. A general two-qubit state $|\psi\rangle$ is a superposition of these four basis states:

$$|\psi\rangle = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle \tag{1.10}$$

where $\sum_{i,j\in\{0,1\}} |\alpha_{ij}|^2 = 1$.

This exponential scaling of the state space dimension (2^n for n qubits) is a primary source of the proposed quantum computational advantage.

1.5 Quantum Entanglement

The tensor product structure naturally gives rise to states that have no classical analog. A multi-qubit state is said to be *separable* (or a product state) if it can be written as the tensor product of individual qubit states. For instance, the state $|\psi\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$ is separable because it can be factored as $\left(\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\right) \otimes \left(\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\right)$.

However, the vast majority of states in a composite Hilbert space cannot be factored in this manner. States that are not separable are called *entangled*. Entanglement represents a profound form of non-local correlation between quantum systems, famously described by Einstein, Podolsky, and Rosen (EPR) as "spooky action at a distance" [EPR35].

1.5.1 The Bell States

The most famous examples of maximally entangled two-qubit states are the four Bell states (or EPR pairs). They form an orthonormal basis for the two-qubit state space and are defined as:

$$\left|\Phi^+\right\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \tag{1.11}$$

$$\left|\Phi^-\right\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle) \tag{1.12}$$

$$\left|\Psi^+\right\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle) \tag{1.13}$$

$$\left|\Psi^-\right\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle) \tag{1.14}$$

Consider the state $|\Phi^+\rangle$. If we measure the first qubit and observe the outcome 0, the state instantaneously collapses to $|00\rangle$.

Consequently, a subsequent measurement of the second qubit will yield 0 with 100% certainty, regardless of the physical distance separating the two qubits.

This perfect correlation violates the Bell inequalities, proving that quantum mechanics cannot be described by local hidden variable theories [Bel64].

Because an entangled state cannot be described by assigning definite independent states to its constituent qubits, the Bloch sphere representation fails entirely for entangled qubits. The state of the global system is completely known, yet the state of the individual subsystems is maximally mixed (completely unknown).

1.6 Quantum Logic Gates and Circuits

Classical algorithms are typically described using combinations of basic Boolean logic gates (such as AND, OR, and NOT). Analogously, quantum algorithms are formalized using the *quantum circuit model*, where computation is executed via a sequence of unitary operations called quantum logic gates applied to a register of qubits, followed by measurements.

Because quantum evolution must be reversible, all quantum logic gates, with the exception of measurement, must be reversible. Consequently, quantum gates are represented by unitary matrices.

1.6.1 Single-Qubit Gates

Single-qubit gates act on the state space $\mathbb{C}^2$ and are represented by 2×2 unitary matrices. The most fundamental of these are the Pauli matrices, which correspond to rotations by π radians around the x, y, and z axes of the Bloch sphere:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \tag{1.15}$$

$$Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \tag{1.16}$$

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \tag{1.17}$$

The X gate acts as a quantum bit-flip, mapping $|0\rangle \to |1\rangle$ and $|1\rangle \to |0\rangle$. The Z gate acts as a phase-flip, leaving $|0\rangle$ unchanged but mapping $|1\rangle \to -|1\rangle$.

Arguably the most ubiquitous single-qubit gate in quantum algorithms is the Hadamard gate, denoted H. It creates an equal superposition when applied to computational basis states:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \tag{1.18}$$

Applying H to $|0\rangle$ yields the state $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, and applying it to $|1\rangle$ yields $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$.

Other essential single-qubit gates include the Phase gate (S) and the $\pi/8$ gate (T), which introduce relative phases:

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}, \quad T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix} \tag{1.19}$$

1.6.2 Multi-Qubit Gates: The CNOT Gate

To generate entanglement and perform conditional logic, gates must act on multiple qubits. The paradigmatic two-qubit gate is the Controlled-NOT (CNOT) gate.

The CNOT gate acts on a control qubit and a target qubit. If the control qubit is $|1\rangle$, it applies an X gate (bit-flip) to the target qubit; if the control is $|0\rangle$, it leaves the target unchanged.

In the computational basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$, where the first qubit is the control,

the matrix representation is:

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \tag{1.20}$$

The CNOT gate is critical because it is the primary mechanism for creating entangled states. For example, applying a Hadamard gate to the first qubit of the state $|00\rangle$ followed by a CNOT gate produces the Bell state $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$.

1.7 Universal Quantum Computation

In classical computing, a set of logic gates is considered *universal* if any arbitrary Boolean function can be constructed using only gates from that set (e.g., the NAND gate alone is universal).

In the quantum regime, an arbitrary unitary transformation requires continuous parameters, implying an infinite number of possible gates.

However, the Solovay-Kitaev theorem states that any quantum algorithm can be approximated to arbitrary accuracy using a finite, discrete set of quantum gates, with only a polylogarithmic overhead in the number of gates [DN05]. A common universal gate set used in fault-tolerant quantum computing is the combination of the single-qubit Hadamard and T gates, along with the two-qubit CNOT gate: $\{H, T, \text{CNOT}\}$.

1.8 Theoretical Machine and Memory Models

Before analyzing the computational complexity of quantum algorithms, we must rigorously define the theoretical machines capable of executing them. While quantum circuits provide a practical and widely used model for algorithm design, formal complexity theory relies heavily on the Quantum Turing Machine (QTM) and specialized memory architectures to properly define resource scaling and asymptotic behavior. This formalization is crucial for extending the classical extended Church-Turing thesis into the quantum regime [BV97].

1.8.1 The Quantum Turing Machine (QTM)

The Quantum Turing Machine, initially conceptualized in the early 1980s, formalized by Deutsch [Deu85], and rigorously analyzed for complexity by Bernstein and Vazirani [BV97], serves as the quantum analogue of the classical probabilistic Turing machine.

Formally, a QTM is defined by a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$, where:

- Q is a finite set of internal states.
- Σ is the input alphabet.
- Γ is the tape alphabet (where $\Sigma \subset \Gamma$ and includes a blank symbol).

- $q_0, q_{accept}, q_{reject} \in Q$ are the initial, accepting, and rejecting states, respectively.

The core distinction lies in the transition function δ. Instead of a deterministic or probabilistic mapping, δ defines a transition amplitude:

$$\delta : Q \times \Gamma \times Q \times \Gamma \times \{L, R\} \to \mathbb{C} \tag{1.21}$$

For a QTM to be physically valid, its global time evolution must be governed by a unitary operator U. Unlike classical machines where any local probabilistic transitions guarantee a valid global evolution, a QTM's local transition amplitudes must satisfy strict well-formedness conditions to ensure global unitarity (e.g., local preservation of the norm and orthogonality of configurations shifting in opposite directions) [BV97].

The state of the QTM is represented by a vector in the Hilbert space $\mathcal{H} = \mathcal{H}_{tape} \otimes \mathcal{H}_{head} \otimes \mathcal{H}_{state}$. If the machine is in a superposition of configurations, the probability of observing a specific configuration upon measurement is the absolute square of its complex amplitude. The restriction that $U^\dagger U = I$ enforces that the computation is strictly reversible and that probability is conserved globally. By demonstrating that a universal QTM can simulate any other QTM with only polynomial overhead, the theoretical foundation for the BQP complexity class was firmly established.

1.8.2 Quantum Random Access Memory (QRAM)

While the QTM encapsulates general computation, certain quantum algorithms—such as the HHL algorithm for linear systems [HHL09] or various quantum machine learning protocols—assume the ability to query classical data in a quantum superposition. This necessitates a Quantum Random Access Memory (QRAM) [GLM08].

A QRAM allows a quantum computer to access memory cells using a quantum superposition of addresses. Given an address register A and a data register D, the QRAM operation applies the unitary transformation:

$$\sum_{i=0}^{N-1} \alpha_i |i\rangle_A |0\rangle_D \xrightarrow{\text{QRAM}} \sum_{i=0}^{N-1} \alpha_i |i\rangle_A |x_i\rangle_D \tag{1.22}$$

where x_i is the classical data stored at the i-th classical memory location.

A naive realization of QRAM would require a "fan-out" architecture, entangling the address register with all N memory cells simultaneously. This leads to massive decoherence because $\mathcal{O}(N)$ quantum gates are actively interacting with the environment during every query.

To resolve this, Giovannetti et al. [GLM08] proposed the *bucket-brigade architecture*. In this model, the memory is structured as a binary routing tree. To access a memory cell, the address qubits are sequentially injected into the root of the tree, dictating the routing path by altering the states of the intermediate nodes (often modeled as qutrits) from a passive "wait" state to active routing states. The crucial advantage is that only the nodes along the specific paths in the superposition are actively queried, while the rest of the tree

remains passive. Consequently, accessing an array of N elements requires only $\mathcal{O}(\log N)$ active quantum switches per query. This exponentially suppresses the decoherence rate compared to the fan-out architecture, making the theoretical model of QRAM physically plausible for fault-tolerant quantum computation.

1.9 Computational Complexity and Quantum Advantage

The classification of problems based on their asymptotic resource scaling (time and space) with respect to input size N allows us to rigorously quantify quantum advantage. By embedding quantum computation within the broader landscape of classical computational complexity [AB09], we establish a formal framework to evaluate the performance of quantum algorithms against their classical counterparts.

1.9.1 Definitions of Complexity Classes

Before establishing the relationships between classical and quantum computation, we must formally define the relevant complexity classes.

- **P (Polynomial Time):** The class of decision problems solvable by a deterministic classical Turing machine in time $\mathcal{O}(N^k)$ for some constant k. This represents the class of problems that are classically tractable.
- **ZPP (Zero-error Probabilistic Polynomial time):** The class of problems solvable by a probabilistic classical Turing machine that never makes an error (returns the correct answer with probability 1), but its running time is polynomial only in *expectation*. Formally, ZPP = RP $\cap$ co-RP.
- **RP (Randomized Polynomial time):** The class of decision problems solvable by a probabilistic Turing machine in polynomial time with one-sided error. If the correct answer is "no", the machine always returns "no". If the correct answer is "yes", it returns "yes" with a probability of at least 1/2.
- **BPP (Bounded-error Probabilistic Polynomial time):** The class of decision problems solvable by a probabilistic classical Turing machine in polynomial time, where the machine yields the correct answer with a probability strictly bounded away from 1/2 (typically defined as $\geq 2/3$) for all inputs. Through standard error amplification (running the algorithm multiple times and taking a majority vote), this error can be made exponentially small [AB09].
- **NP (Nondeterministic Polynomial time):** The class of decision problems for which a "yes" instance possesses a classical witness (or certificate) that can be verified by a deterministic Turing machine in polynomial time.
- **BQP (Bounded-error Quantum Polynomial time):** The formal quantum analogue of BPP. It is the class of decision problems solvable by a QTM, or a uniform

family of polynomial-size quantum circuits, in polynomial time, with an error probability bounded by 1/3 for all inputs [BV97; NC10].

- **PP (Probabilistic Polynomial time):** The class of decision problems solvable by a probabilistic Turing machine in polynomial time where the probability of returning the correct answer is strictly greater than 1/2. Because the error probability is not bounded away from 1/2 by a constant, PP is a remarkably powerful class that contains both NP and BQP [ADH97].
- **PSPACE (Polynomial Space):** The class of decision problems solvable by a deterministic Turing machine using strictly $\mathcal{O}(N^k)$ memory space, without any restriction on execution time.

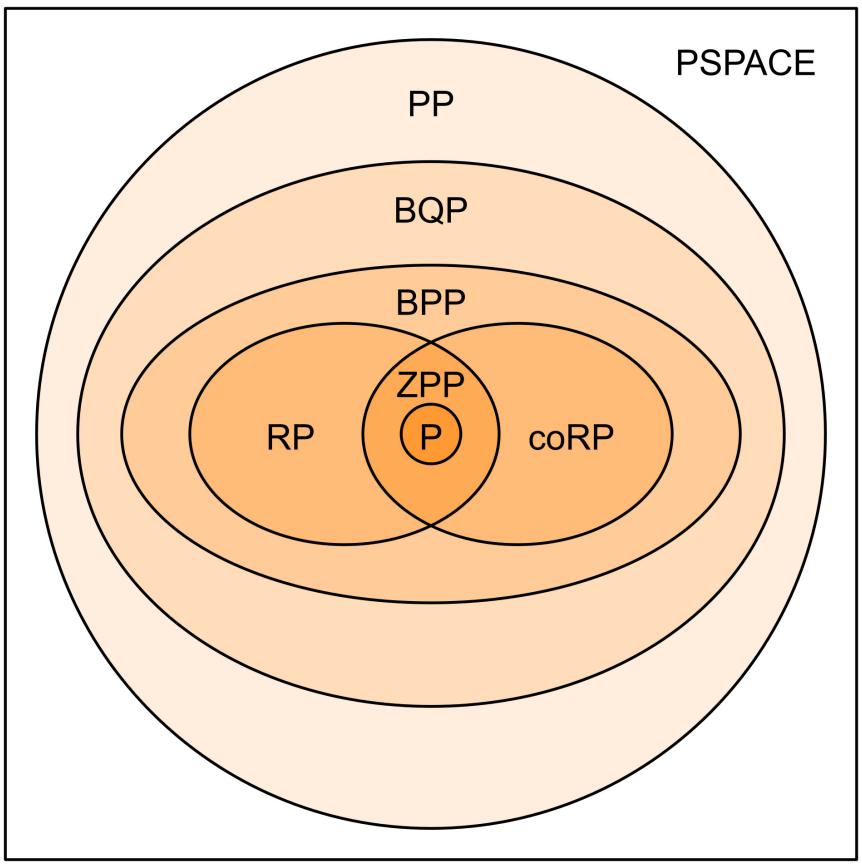

Figure 1.2: Euler diagram illustrating the theoretical relationships and hierarchy of classical and quantum computational complexity classes. Source: Wikipedia

1.9.2 Proofs of Inclusion

To contextualize quantum advantage, we formally establish the hierarchy of inclusions: P $\subseteq$ ZPP $\subseteq$ RP $\subseteq$ BPP $\subseteq$ BQP $\subseteq$ PP $\subseteq$ PSPACE. While it is widely conjectured that

BQP is strictly larger than BPP, definitively proving strict inclusions remains one of the most significant open problems in theoretical computer science.

Theorem 1: $\mathrm{P} \subseteq \mathrm{BPP}$

Proof. Let L be a language such that $L \in \mathrm{P}$. By definition, there exists a deterministic Turing machine M that decides whether an input $x \in L$ in polynomial time with 0 error probability. We can construct a probabilistic Turing machine M' that simply ignores its capability to generate random coin tosses and perfectly executes the deterministic transition logic of M. Because M' operates exactly as M, it outputs the correct answer with a probability of 1. Since 1 is strictly greater than the 2/3 threshold required for BPP, it follows immediately that $L \in \mathrm{BPP}$. □

Theorem 2: $\mathrm{BPP} \subseteq \mathrm{BQP}$

Proof. Let $L \in \mathrm{BPP}$. There exists a probabilistic classical Turing machine M that decides L in polynomial time $p(N)$ with an error probability bounded by 1/3. To show that $L \in \mathrm{BQP}$, we must simulate M using a Quantum Turing Machine (QTM) in polynomial time. This requires two logical steps:

Step 1: Classical Reversible Simulation. According to Bennett's theorem [Ben73], any classical computation can be made reversible with at most a polynomial overhead in time and space. We can replace all irreversible classical logic gates (like AND/OR) with reversible quantum equivalents (like the Toffoli gate).

Thus, we construct a deterministic, reversible quantum circuit C_{rev} that mimics the deterministic parts of M.

Step 2: Quantum Randomness Generation. The machine M requires random bits to function.

In a quantum circuit, a perfectly unbiased random bit can be generated by applying a Hadamard gate H to an ancilla qubit initialized in the $|0\rangle$ state:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \tag{1.23}$$

Measuring this state in the computational basis yields a 0 or 1, each with a probability of $|\frac{1}{\sqrt{2}}|^2 = 1/2$.

By using polynomial applications of the Hadamard gate to supply random bits to the reversible circuit C_{rev}, the QTM perfectly simulates the probabilistic logic of M.

Because the internal state transitions map exactly to the classical probabilistic paths, the QTM decides L in polynomial time with the exact same error bound ($\leq 1/3$). Therefore, $L \in \mathrm{BQP}$. □

Theorem 3: $\mathrm{BQP} \subseteq \mathrm{PSPACE}$ (Direct Proof)

Proof. Let $L \in \mathrm{BQP}$.

By definition, there exists a uniform family of quantum circuits $\{C_N\}$ of size $m = \mathcal{O}(N^k)$ acting on $n = \mathcal{O}(N^j)$ qubits that decides L. To prove $L \in \mathrm{PSPACE}$, we must show

that a deterministic classical machine can calculate the final probability amplitude of the accepting state using only polynomial memory [BV97].

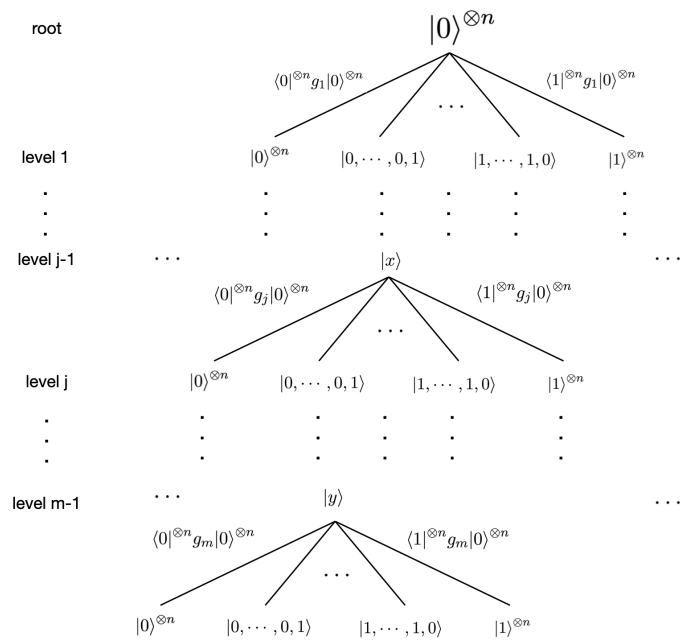

Figure 1.3: A computational tree representing the Feynman sum-over-histories approach for evaluating the probability amplitudes of quantum circuits. Source: Wikipedia

Let the circuit C_N consist of m sequential unitary gates $U_1, U_2, \ldots, U_m$. The final transition amplitude to a specific basis state $|y\rangle$ from an initial state $|x\rangle$ is given by:

$$\langle y|U_m U_{m-1} \ldots U_1|x\rangle \tag{1.24}$$

By inserting the resolution of the identity $I = \sum_z |z\rangle\langle z|$ between every gate operation, we can rewrite this amplitude as a Feynman sum over all possible classical computational paths:

$$\sum_{z_1, z_2, \ldots, z_{m-1}} \langle y|U_m|z_{m-1}\rangle\langle z_{m-1}|U_{m-1}|z_{m-2}\rangle \ldots \langle z_1|U_1|x\rangle \tag{1.25}$$

Because there are n qubits, there are 2^n possible basis states at each of the m steps. Consequently, evaluating this sum explores 2^{nm} distinct paths, which requires exponential time.

However, a deterministic classical machine can evaluate this sum in polynomial *space* using a depth-first search of the computational tree. The machine evaluates one single path at a time. To do this, it only needs to store:

1. The current path being evaluated, which requires $m \times n$ bits (a polynomial amount of space).

2. A running total of the cumulative complex amplitude. Since BQP allows an error

margin of 1/3, we do not need infinite precision. Storing the running total to polynomial precision (polynomial number of bits) is sufficient to boundedly approximate the final probability.

Once the amplitude of a single path is computed and added to the running total, the memory used to store that path is overwritten by the next path in the sequence.

Because the depth of the tree is bounded by m (a polynomial), and the precision required is bounded by a polynomial, the maximum memory utilized at any given moment scales polynomially with N.

Thus, the entire simulation operates within PSPACE, proving $L \in$ PSPACE. □

Theorem 4: BQP $\subseteq$ PP

Proof. Let $L \in$ BQP. By definition, there exists a uniform family of polynomial-size quantum circuits that decides L with an error probability bounded by 1/3.

Computing the exact acceptance probability involves summing over exponentially many classical paths, identical to the Feynman path formulation in Theorem 3. A PP machine can nondeterministically guess a computational path and verify it. Because PP allows for an unbounded error margin (the acceptance threshold only needs to be strictly greater than 1/2), the PP machine can encode the positive and negative contributions of the complex transition amplitudes into its acceptance and rejection paths. By carefully balancing the number of accepting paths to reflect the squared amplitudes of the quantum state, the PP machine will accept with a probability $> 1/2$ if and only if the quantum circuit accepts with probability $\geq 2/3$. Thus, the polynomial-time quantum computation is subsumed within the classical probabilistic polynomial-time framework with unbounded error, proving $L \in$ PP [ADH97].

□

Theorem 5: PP $\subseteq$ PSPACE

Proof. Let $L \in$ PP. A probabilistic Turing machine M deciding L runs in polynomial time, meaning it tosses at most a polynomial number of coins, $m = \mathcal{O}(N^k)$, and thus has at most 2^m possible execution paths.

To determine if the acceptance probability is strictly greater than 1/2, a deterministic Turing machine can systematically enumerate every possible path of M. By using a depth-first search (similar to Theorem 3), the deterministic machine only needs to store the current path of length m and maintain a counter of the number of accepting paths.

Since the counter maxes out at 2^m, it requires exactly m bits of space. Because the maximum memory utilized at any moment scales polynomially with N, the entire simulation operates within PSPACE. Therefore, $L \in$ PSPACE. □

Bibliography

[EPR35] Albert Einstein, Boris Podolsky, and Nathan Rosen. "Can quantum-mechanical description of physical reality be considered complete?" In: *Physical review* 47.10 (1935), p. 777.

[Bel64] John S Bell. "On the Einstein Podolsky Rosen paradox". In: *Physics Physique Fizika* 1.3 (1964), p. 195.

[Ben73] Charles H Bennett. "Logical reversibility of computation". In: *IBM journal of Research and Development* 17.6 (1973), pp. 525–532.

[Ben80] Paul Benioff. "The computer as a physical system: A microscopic quantum mechanical Hamiltonian model of computers as represented by Turing machines". In: *Journal of statistical physics* 22.5 (1980), pp. 563–591.

[Fey82] Richard P Feynman. "Simulating physics with computers". In: *International journal of theoretical physics* 21.6-7 (1982), pp. 467–488.

[Deu85] David Deutsch. "Quantum theory, the Church-Turing principle and the universal quantum computer". In: *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences* 400.1818 (1985), pp. 97–117.

[Sho94] Peter W Shor. "Algorithms for quantum computation: discrete logarithms and factoring". In: *Proceedings 35th annual symposium on foundations of computer science*. Ieee. 1994, pp. 124–134.

[ADH97] Leonard M Adleman, Jonathan DeMarrais, and Ming-Deh Huang. "Quantum computability". In: *SIAM Journal on Computing* 26.5 (1997), pp. 1524–1540.

[BV97] Ethan Bernstein and Umesh Vazirani. "Quantum complexity theory". In: *SIAM Journal on computing* 26.5 (1997), pp. 1411–1473.

[DN05] Christopher M Dawson and Michael A Nielsen. "The Solovay-Kitaev algorithm". In: *arXiv preprint quant-ph/0505030* (2005).

[GLM08] Vittorio Giovannetti, Seth Lloyd, and Lorenzo Maccone. "Quantum random access memory". In: *Physical review letters* 100.16 (2008), p. 160501.

[AB09] Sanjeev Arora and Boaz Barak. *Computational complexity: a modern approach*. Cambridge University Press, 2009.

[HHL09] Aram W Harrow, Avinatan Hassidim, and Seth Lloyd. "Quantum algorithm for linear systems of equations". In: *Physical review letters* 103.15 (2009), p. 150502.

[NC10] Michael A Nielsen and Isaac L Chuang. *Quantum computation and quantum information*. Cambridge University Press, 2010.